

Background I

Overview of Some Deep Learning Libraries

Machine learning is a broad topic. Deep learning, in particular, is a way of using neural networks for machine learning. A neural network is probably a concept older than machine learning, dating back to the 1950s. Unsurprisingly, there were many libraries created for it.

The following aims to give an overview of some of the famous libraries for neural networks and deep learning. After finishing this chapter, you will learn

- ▷ Some of the deep learning or neural network libraries
- ▷ The functional difference between two common libraries, PyTorch and TensorFlow

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ The C++ Libraries
- ▷ Python Libraries
- ▷ PyTorch and TensorFlow

1.1 The C++ Libraries

Deep learning has gained attention in the last decade. Before that, there was little confidence in how to train a neural network with many layers. However, understanding how to build a multilayer perceptron was around for many years.

Before we had deep learning, probably the most famous neural network library was `libann`. It is a library for C++, and the functionality is limited due to its age. This library has stopped development since 2004. A newer library for C++ is `OpenNN`, which allows modern C++ syntax.

But that's pretty much all for C++. The rigid syntax of C++ may be why we do not have too many libraries for deep learning. The training phase of a deep learning project is

about experiments. We want some tools that allow us to iterate faster. Hence a dynamic programming language could be a better fit. Therefore, you see Python come on the scene.

1.2 Python Libraries

One of the earliest libraries for deep learning is Caffe. It was developed at U.C. Berkeley specifically for computer vision problems. While it is developed in C++, it serves as a library with a Python interface. Hence we can build our project in Python with the network defined in a JSON-like syntax.

Chainer is another library in Python. It is an influential one because the syntax makes a lot of sense. While it is less common nowadays, the API in Keras and PyTorch bears a resemblance to Chainer. The following is an example from Chainer's documentation, and you may mistake it as Keras or PyTorch:

```
import chainer
import chainer.functions as F
import chainer.links as L
from chainer import iterators, optimizer, training, Chain
from chainer.datasets import mnist

train, test = mnist.get_mnist()
batchsize = 128
max_epoch = 10

train_iter = iterators.SerialIterator(train, batchsize)

class MLP(Chain):
    def __init__(self, n_mid_units=100, n_out=10):
        super(MLP, self).__init__()
        with self.init_scope():
            self.l1 = L.Linear(None, n_mid_units)
            self.l2 = L.Linear(None, n_mid_units)
            self.l3 = L.Linear(None, n_out)

    def forward(self, x):
        h1 = F.relu(self.l1(x))
        h2 = F.relu(self.l2(h1))
        return self.l3(h2)

# create model
model = MLP()
model = L.Classifier(model) # using softmax cross entropy

# set up optimizer
optimizer = optimizers.MomentumSGD()
optimizer.setup(model)

# connect train iterator and optimizer to an updater
updater = training.updaters.StandardUpdater(train_iter, optimizer)
```

```
# set up trainer and run
trainer = training.Trainer(updater, (max_epoch, 'epoch'), out='mnist_result')
trainer.run()
```

Listing 1.1: Example Chainer code

The other obsoleted library is Theano. It has ceased development, but once upon a time, it was a major library for deep learning. The earlier version of the Keras library allows you to choose between a Theano or TensorFlow backend. Indeed, neither Theano nor TensorFlow are deep learning libraries precisely. Rather, they are tensor libraries that make matrix operations and differentiation handy, upon where deep learning operations can be built. Hence these two are considered replacements for each other from Keras' perspective.

CNTK from Microsoft and Apache MXNet are two other libraries that worth mentioning. They are large with interfaces for multiple languages. Python, of course, is one of them. CNTK has C# and C++ interfaces, while MXNet provides interfaces for Java, Scala, R, Julia, C++, Clojure, and Perl. But recently, Microsoft decided to stop developing CNTK. MXNet does have some momentum, and it is probably the most popular library after TensorFlow and PyTorch.

Below is an example of using MXNet via the R interface. Conceptually, you see the syntax is similar to Keras' functional API:

```
require(mxnet)

train <- read.csv('data/train.csv', header=TRUE)
train <- data.matrix(train)
train.x <- train[,-1]
train.y <- train[,1]
train.x <- t(train.x/255)

data <- mx.symbol.Variable("data")
fc1 <- mx.symbol.FullyConnected(data, name="fc1", num_hidden=128)
act1 <- mx.symbol.Activation(fc1, name="relu1", act_type="relu")
fc2 <- mx.symbol.FullyConnected(act1, name="fc2", num_hidden=64)
act2 <- mx.symbol.Activation(fc2, name="relu2", act_type="relu")
fc3 <- mx.symbol.FullyConnected(act2, name="fc3", num_hidden=10)
softmax <- mx.symbol.SoftmaxOutput(fc3, name="sm")

devices <- mx.cpu()
mx.set.seed(0)
model <- mx.model.FeedForward.create(softmax, X=train.x, y=train.y,
                                     ctx=devices, num.round=10, array.batch.size=100,
                                     learning.rate=0.07, momentum=0.9,
                                     eval.metric=mx.metric.accuracy,
                                     initializer=mx.init.uniform(0.07),
                                     epoch.end.callback=mx.callback.log.train.metric(100))
```

Listing 1.2: Example MXNet code in R

1.3 PyTorch and TensorFlow

PyTorch and TensorFlow are the two major libraries nowadays. In the past, when TensorFlow was in version 1.x, they were vastly different. But since TensorFlow absorbed Keras as part of its library, these two libraries mostly work similarly.

PyTorch is backed by Facebook, and its syntax has been stable over the years. There are also a lot of existing models that we can borrow. The common way of defining a deep learning model in PyTorch is to create a class:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2)
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.flatten = nn.Flatten()
        self.linear4 = nn.Linear(120, 84)
        self.linear5 = nn.Linear(84, 10)
        self.softmax = nn.LogSoftMax(dim=1)

    def forward(self, x):
        x = F.tanh(self.conv1(x))
        x = self.pool1(x)
        x = F.tanh(self.conv2(x))
        x = self.pool2(x)
        x = F.tanh(self.conv3(x))
        x = self.flatten(x)
        x = F.tanh(self.linear4(x))
        x = self.linear5(x)
        return self.softmax(x)

model = Model()
```

Listing 1.3: Example PyTorch code using sub-classing syntax

But there is also a sequential syntax to make the code more concise:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    # assume input 1x28x28
    nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
```

```

nn.Tanh(),
nn.AvgPool2d(kernel_size=2, stride=2),
nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0),
nn.Tanh(),
nn.Flatten(),
nn.Linear(120, 84),
nn.Tanh(),
nn.Linear(84, 10),
nn.LogSoftmax(dim=1)
)

```

Listing 1.4: Example PyTorch code using sequential syntax

TensorFlow in version 2.x adopted Keras as part of its libraries. In the past, these two were separate projects. In TensorFlow 1.x, we need to build a computation graph, setup a session, and derive gradients from a session for the deep learning model. Hence it is a bit too verbose. Keras is designed as a library to hide all these low-level details.

The same network as above can be produced by TensorFlow's Keras syntax as follows:

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, AveragePooling2D, Flatten

model = Sequential([
    Conv2D(6, (5,5), input_shape=(28,28,1), padding="same", activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(16, (5,5), activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(120, (5,5), activation="tanh"),
    Flatten(),
    Dense(84, activation="tanh"),
    Dense(10, activation="softmax")
])

```

Listing 1.5: Example TensorFlow code using sequential syntax

One major difference between PyTorch and Keras syntax is in the training loop. In Keras, we just need to assign the loss function, the optimization algorithm, the dataset, and some other parameters to the model. Then we have a `fit()` function to do all the training work, as follows:

```

...
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=100, batch_size=32)

```

Listing 1.6: Using `fit()` function in TensorFlow

But in PyTorch, we need to write our own training loop code:

```

# self-defined training loop function
def training_loop(model, optimizer, loss_fn, train_loader, val_loader=None, n_epochs=100):
    best_loss, best_epoch = np.inf, -1
    best_state = model.state_dict()

```

```

for epoch in range(n_epochs):
    # Training
    model.train()
    train_loss = 0
    for data, target in train_loader:
        output = model(data)
        loss = loss_fn(output, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
    # Validation
    model.eval()
    status = (f"{str(datetime.datetime.now())} End of epoch {epoch}, "
              f"training loss={train_loss/len(train_loader)}")
    if val_loader:
        val_loss = 0
        for data, target in val_loader:
            output = model(data)
            loss = loss_fn(output, target)
            val_loss += loss.item()
        status += f", validation loss={val_loss/len(val_loader)}"
    print(status)

optimizer = optim.Adam(model.parameters())
criterion = nn.NLLLoss()
training_loop(model, optimizer, criterion, train_loader, test_loader, n_epochs=100)

```

Listing 1.7: User-defined training loop function for PyTorch

This may not be an issue if you're experimenting with a new design of network in which you want to have more control over how the loss is calculated and how the optimizer updates the model weights. But otherwise, you will appreciate the simpler syntax from Keras.

Note that both PyTorch and TensorFlow are libraries with a Python interface. Therefore, it is possible to have an interface for other languages too. For example, there are Torch for R and TensorFlow for R.

Also, note that the libraries we mentioned above are full-featured libraries that include training and prediction. If you consider a production environment where you make use of a trained model, there could be a wider choice. TensorFlow has a "TensorFlow Lite" counterpart that allows a trained model to be run on a mobile device or the web. Intel also has an OpenVINO library that aims to optimize the performance in prediction.

1.4 Further Reading

Below are the links to the libraries we mentioned above:

libann.

<https://www.nongnu.org/libann/>

OpenNN.

<https://www.opennn.net/>

Chainer.

<https://chainer.org/>

Caffe.

<https://caffe.berkeleyvision.org/>

CNTK.

<https://docs.microsoft.com/en-us/cognitive-toolkit/>

MXNet.

<https://mxnet.apache.org/versions/1.9.1/>

Theano.

<https://github.com/Theano/Theano>

PyTorch.

<https://pytorch.org/>

TensorFlow.

<https://www.tensorflow.org/>

Torch for R.

<https://torch.mlverse.org/>

TensorFlow for R.

<https://tensorflow.rstudio.com/>

1.5 Summary

In this chapter, you discovered various deep learning libraries and some of their characteristics. Specifically, you learned:

- ▷ What are the libraries available for C++ and Python
- ▷ How the Chainer library influenced the syntax in building a deep learning model
- ▷ The relationship between Keras and TensorFlow 2.x
- ▷ What are the differences between PyTorch and TensorFlow

In the next chapter, you will learn more about the library of choice in this book, PyTorch.

Introduction to PyTorch

PyTorch is an open-source machine learning library developed by Facebook's AI Research team. It is a Python-based library used for building deep neural networks and supports dynamic computation graphs. In the following, you will learn about this amazing library and what it can do at a high level. After finishing this chapter, you will know:

- ▷ What are the main features of PyTorch to make it a deep learning library
- ▷ How to install PyTorch
- ▷ Some basic examples of PyTorch code

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ PyTorch at a High Level
- ▷ Applications of PyTorch
- ▷ Advantages and Limitations of PyTorch
- ▷ Comparing PyTorch to TensorFlow

2.1 PyTorch at a High Level

PyTorch is a library that provides two important high-level features:

1. Tensor computation (like NumPy), optionally with acceleration via graphics processing units (GPU).
2. Deep neural networks built on automatic differentiation

PyTorch is a deep learning library because of these two key features. *Tensor* is a general term for arrays and matrices. Components in a deep learning model are represented as matrices and matrix formulas mathematically. The tensor computation allows numbers in matrices to be manipulated concisely. GPU is sometimes used to speed up the matrix computation. A deep learning model can run (in *inference mode* or *forward pass*) with tensor computation,

but you need automatic differentiation to train the model or run the *backward pass*. These two features together allow us to develop deep learning models in PyTorch.

PyTorch has become very popular among machine learning enthusiasts due to its simplicity and ease of use. It is also gaining popularity in the research community due to its dynamic nature, which allows researchers to change the model architecture on the fly.

Some key features of PyTorch include:

- ▷ A simple and intuitive API that allows developers to quickly prototype their models.
- ▷ Seamless integration with NumPy, making it easy to move data between the two libraries.
- ▷ Dynamic computation graphs that allow for easier debugging and faster experimentation.
- ▷ Built-in support for distributed training across multiple GPUs and machines.
- ▷ A large and active community that provides support and contributes to the development of the library.

In summary, PyTorch is a powerful and easy-to-use deep learning library that is widely used in both academia and industry. Its dynamic nature, ease of use, and strong community support make it an excellent choice for anyone looking to get started with deep learning.

2.2 Getting Started with PyTorch

To get started with PyTorch, you first need to install it. You can do this easily using `pip` by running the following command:

```
pip install torch
```

Listing 2.1: Installing PyTorch using `pip`

If you're using `conda`, the package name is different and the command would be:

```
conda install pytorch
```

Listing 2.2: Installing PyTorch using `conda`

This is the simplest, but you may also need to install CUDA library before your PyTorch installation can run on GPU. Once you have installed PyTorch, you can start experimenting with it. Here is a simple example to let you verify your PyTorch is working:

```
import torch

# create a tensor of size (2,3)
x = torch.tensor([[1, 2, 3], [4, 5, 6]])

# create a tensor of size (3,2)
y = torch.tensor([[1, 2], [3, 4], [5, 6]])
```

```
# matrix multiplication of x and y
z = torch.matmul(x, y)

print(z)
```

Listing 2.3: A simple PyTorch test program

This will output:

```
tensor([[22, 28],\
        [49, 64]])
```

Output 2.1: Output from the test program

This is just a simple example, but it demonstrates how PyTorch performs tensor computation.

2.3 Applications of PyTorch

PyTorch is widely used in various applications of machine learning and deep learning such as:

- ▷ **Computer Vision:** PyTorch is used in computer vision tasks such as image classification, object detection, and segmentation. Some popular computer vision models such as ResNet, YOLO, and Faster R-CNN have a PyTorch implementation.
- ▷ **Natural Language Processing (NLP):** PyTorch is used in NLP tasks such as sentiment analysis, machine translation, and text generation. Some popular NLP models including BERT and Transformer have a PyTorch implementation.
- ▷ **Reinforcement Learning:** PyTorch is also used in reinforcement learning tasks such as game playing and robot control. Some popular reinforcement learning models such as DQN, PPO, and A3C have a PyTorch implementation.
- ▷ **Other Applications:** Apart from the above-mentioned applications, PyTorch is also used in other areas such as speech recognition, time series forecasting, and anomaly detection.

2.4 Advantages and Limitations of PyTorch

PyTorch has several advantages as a deep learning library:

- ▷ **Dynamic Computation Graphs:** PyTorch uses dynamic computation graphs, which means that the graph is generated on the fly as the operations are executed. This allows you to code at high level without worrying how the computer should execute it. In other words, you can experiment faster.
- ▷ **Easy to Use:** PyTorch has a simple and intuitive API that allows developers to quickly prototype their models. It is also easy to install and setup.
- ▷ **GPU Acceleration** PyTorch supports strong acceleration via graphics processing units (GPU), making it faster than some other deep learning libraries.

- ▷ **Active Community:** PyTorch has a large and active community that provides support and contributes to the development of the library. This means that users can quickly get help and access to new features.

PyTorch has some limitations that users should be aware of:

- ▷ **Limited Production Use** PyTorch is mainly used for research and prototyping. It is not yet widely used in production environments.
- ▷ **Steep Learning Curve** Although PyTorch has a simple API, it can still have a steep learning curve, especially for users who are new to deep learning.

2.5 Comparing PyTorch to TensorFlow

PyTorch and TensorFlow are considered rivals. They have different design but similar in functionality.

TensorFlow has a more primitive interface that you can drill down to a lower level than what PyTorch can provide. However, when developing a deep learning model, you would prefer to work at conceptual levels such as layers and optimization algorithms rather than arithmetic of a single value. Therefore, in TensorFlow 2, it is tightly coupled with the high level library Keras. You will find Keras and PyTorch syntax are comparable as both allows you to design a deep learning model at a high level.

However, you will find Keras influenced by the design of an older library scikit-learn that sometimes it hid too much detail. For example, in TensorFlow and Keras, you normally train a model with a function call `fit()` which all the automatic differentiation and gradient update are done implicitly. In PyTorch, you need to write lines of code to clearly specify how exactly these are done. In cases that you are innovative on the details of how a deep learning model is trained, you will find PyTorch more flexible. This is also one of the reasons why PyTorch is popular in research communities.

In detail, there are some subtle design choice that make one library better than another. In Keras, for example, there is a concept of Keras tensor which is distinct from a tensor from TensorFlow that works as a placeholder. This is sometimes confusing to consider whether a tensor should contain value or not. PyTorch doesn't use such placeholder tensors to build a model. But the downside of this is each component in a deep learning model are disconnected. The logic of how a tensor *flows* between components is unclear until the model actually runs. You will see this creates trouble when you want to convert a model into a picture, which Keras has a `plot_model()` function and no PyTorch counterpart exists.

The other aspect that set these two libraries apart are the family of their sister projects. PyTorch has a torchvision library that is specific for computer vision models. You will find a lot of useful image processing routines in the library that make your computer vision project much easier. TensorFlow, on the other hand, has very rich mobile and edge device support. You can easily convert a TensorFlow model into a TensorFlow.js model to run as JavaScript on a browser, or into a TensorFlow Lite model that can run on memory-limited mobile and edge devices. The TensorBoard project that provides a dashboard to visualize various aspects of a model and its training is appreciated by many. It is so helpful that PyTorch even provided built-in support to write data to TensorBoard.

2.6 Further Readings

It is easy to find books on PyTorch, but the most current documentation is always from PyTorch's official website:

PyTorch.

<https://pytorch.org/>

PyTorch documentation.

<https://pytorch.org/docs/stable/index.html>

PyTorch tutorials.

<https://pytorch.org/tutorials/>

2.7 Summary

PyTorch is a powerful and easy-to-use deep learning library that is widely used in both academia and industry. Its dynamic nature, ease of use, and strong community support make it an excellent choice for anyone looking to get started with deep learning. PyTorch is widely used in various applications such as computer vision, NLP, and reinforcement learning. It has several advantages over other deep learning libraries, such as dynamic computation graphs, easy-to-use API, GPU acceleration, and an active community. However, it also has some limitations, such as limited production use and relatively steep learning curve, PyTorch is an excellent choice for anyone looking to get started with deep learning.

In the next chapter, you will learn how to use PyTorch as a tensor library to manipulate an array of numbers.

Manipulating Tensors in PyTorch

PyTorch is a deep-learning library. Just like some other deep learning libraries, it applies operations on numerical arrays called *tensors*. In the simplest terms, tensors are just multidimensional arrays. When we deal with the tensors, some operations are used very often. In PyTorch, there are some functions defined specifically for dealing with tensors.

In the following, we will give a brief overview of what PyTorch provides on tensors and how we can use them. After finishing this chapter, you will know:

- ▷ How to create and operate on PyTorch tensors
- ▷ PyTorch's tensor syntax is similar to NumPy
- ▷ The common functions you can use from PyTorch to manipulate a tensor

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Creating Tensors
- ▷ Checking a Tensor
- ▷ Manipulating Tensors
- ▷ Tensor Functions

3.1 Creating Tensors

If you're familiar with NumPy, you should recall that there are multiple ways of creating an array. The same is true in PyTorch for creating tensors. The simplest way to create a specific constant matrix like the following:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

is by using:

```
import torch
a = torch.tensor([[1,2,3], [4,5,6]], dtype=torch.int32)
print(a)
```

Listing 3.1: Creating a tensor in PyTorch

It prints:

```
tensor([[1, 2, 3],
        [4, 5, 6]], dtype=torch.int32)
```

Output 3.1: A PyTorch tensor

The `dtype` argument specifies the data type of the values in the tensor. It is optional. You can also provide the values from a NumPy array and convert it to a PyTorch tensor.

Usually, you would create a tensor for some specific purpose. For example, if you want to have ten values evenly distributed between -1 and 1, you can use the `linspace()` function:

```
import torch
a = torch.linspace(-1, 1, 10)
print(a)
```

Listing 3.2: Creating a tensor in PyTorch

It prints:

```
tensor([-1.0000, -0.7778, -0.5556, -0.3333, -0.1111,  0.1111,  0.3333,  0.5556,
         0.7778,  1.0000])
```

Output 3.2: Tensor as produced by `linspace()`

However, if you want to have a tensor of random values (which is very useful in testing your functions), you can make one like the following:

```
import torch
a = torch.rand(3,4)
print(a)
```

Listing 3.3: Creating a random tensor

It prints, for example:

```
tensor([[0.4252, 0.1029, 0.9858, 0.7502],
        [0.1993, 0.6412, 0.2424, 0.6451],
        [0.7878, 0.7615, 0.9170, 0.8534]])
```

Output 3.3: A random tensor of shape 3×4

This resulting tensor is of dimension 3×4 , and each value is uniformly distributed between 0 and 1. If you want to have the values normally distributed, just change the function to `randn()`:

```
a = torch.randn(3,4)
```

Listing 3.4: Creating a tensor of normally distributed random values

If you want to have the random values be an integer, e.g., between 3 to 10, you can use the `randint()` function:

```
import torch
a = torch.randint(3, 10, size=(3,4))
print(a)
```

Listing 3.5: Creating a tensor of random integers

This will give, for example:

```
tensor([[4, 5, 7, 9],
        [3, 8, 8, 9],
        [4, 7, 7, 6]])
```

Output 3.4: A tensor of random integers

The values are in the range $3 \leq x < 10$. By default, the lower bound is zero, so if you want the values to be $0 \leq x < 10$, you can use:

```
a = torch.randint(10, size=(3,4))
```

Listing 3.6: Creating a tensor of random integers

The other commonly used tensors are the zero tensor and tensors with all values the same. To create a zero tensor (e.g., of dimension $2 \times 3 \times 4$), you can use:

```
import torch
a = torch.zeros(2, 3, 4)
print(a)
```

Listing 3.7: Creating a zero tensor

It prints:

```
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],
        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]])
```

Output 3.5: A zero tensor

And to create a tensor of all values are 5, you can use:

```
import torch
a = torch.full((2,3,4), 5)
print(a)
```

Listing 3.8: Creating a full tensor

It prints:

```
tensor([[[[5, 5, 5, 5],
          [5, 5, 5, 5],
          [5, 5, 5, 5]],

        [[5, 5, 5, 5],
          [5, 5, 5, 5],
          [5, 5, 5, 5]]]])
```

Output 3.6: A full tensor is a tensor of identical values

But if you want all values to be one, there is a simpler function:

```
a = torch.ones(2,3,4)
```

Listing 3.9: Creating a tensor of 1's

Finally, if you want an identity matrix, you can get it with `diag()` or `eye()`:

```
import torch
a = torch.eye(4)
print(a)
```

Listing 3.10: Creating an identity tensor

It prints:

```
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])
```

Output 3.7: An identity matrix

3.2 Checking a Tensor

Once you have a tensor and you want to know more about it, you can simply print it to the screen using `print()`. But if the tensor is too big, it is easier to show its dimension by checking its shape:

```
import torch
a = torch.zeros(2, 3, 4)
print(a.shape)
print(a.size())
```

Listing 3.11: Checking tensor shape

It prints:

```
torch.Size([2, 3, 4])
torch.Size([2, 3, 4])
```

Output 3.8: Tensor shape

The shape of a tensor can be accessed using the `shape` property or the `size()` function. If you want to see how many dimensions you have (i.e., $2 \times 3 \times 4$ is 3 and 3×4 is 2), you can read the `ndim` property:

```
print(a.ndim)
```

Listing 3.12: Find the number of dimensions of a tensor

This will give you “3”. If you use `len()` to check a tensor, it will only give you the size of the first dimension, e.g.:

```
import torch
a = torch.zeros(2, 3, 4)
print(len(a))
```

Listing 3.13: Find the size of the first dimension

It prints:

```
2
```

Output 3.9: Size of the first dimension

Another property that you want to learn about a tensor is its data type. Usually, you use floating points in deep learning, but sometimes, the tensors should be in integers (e.g., in an image as pixel values). To check the data type, you can read the `dtype` property:

```
print(a.dtype)
```

Listing 3.14: Find the data type of a tensor

It prints:

```
torch.float32
```

Output 3.10: The data type of a tensor

If you want to change the data type, you can recreate the tensor with a new type:

```
import torch
a = torch.zeros(2, 3, 4)
b = a.type(torch.int32)
print(a.dtype)
print(b.dtype)
```

Listing 3.15: Changing the data type of a tensor is to create another one

The above prints:

```
torch.float32
torch.int32
```

Output 3.11: Data type of the tensors as printed

3.3 Manipulating Tensors

One common operation on tensors in deep learning is to change the tensor shape. For example, you may want to convert a 2D tensor into 1D or add a dummy dimension to a tensor. You may also want to extract a sub-tensor from a larger tensor.

For example, you can create a tensor like the following:

```
a = torch.randn(3,4,5)
print(a)
```

Listing 3.16: Creating a random tensor

If you get:

```
tensor([[[[-1.1271e-01, -7.1124e-01,  1.1335e+00, -8.5644e-01, -1.4191e+00],
          [-1.9065e+00, -6.8386e-02,  5.8727e-01,  6.5890e-03, -2.6947e-01],
          [ 6.3194e-01, -7.7426e-01,  1.6546e+00,  1.2647e-01, -1.0944e+00],
          [ 3.7819e-01, -8.8670e-01,  5.3772e-01,  1.4985e+00,  5.8396e-01]],

        [[ 1.8704e+00,  2.0914e+00, -9.1604e-01,  1.2317e+00, -1.5722e-01],
          [ 2.4689e-01, -2.3157e-01, -3.3033e-01,  1.4021e+00, -6.9540e-01],
          [ 3.0298e-01, -1.4936e-01, -6.8863e-01,  1.6977e-01,  2.4682e+00],
          [-8.1375e-01,  4.8489e-01, -1.2024e+00, -4.9771e-01,  1.1728e-01]],

        [[-1.6011e+00,  1.5686e-03, -1.0560e-01, -1.2938e+00,  5.3077e-01],
          [-9.7636e-01, -9.1854e-01, -1.0002e+00,  1.1852e+00,  1.0328e+00],
          [ 9.6664e-01,  5.3752e-01, -3.1340e-02, -6.7852e-02, -7.2317e-01],
          [-5.5263e-01,  9.4754e-01, -5.4503e-01,  6.3850e-02,  1.2879e+00]]])
```

Output 3.12: A random tensor

It allows you to take a slice using the same syntax as in NumPy:

```
print(a[1])
```

Listing 3.17: Slicing a tensor

This will be:

```
tensor([[ 1.8704,  2.0914, -0.9160,  1.2317, -0.1572],
        [ 0.2469, -0.2316, -0.3303,  1.4021, -0.6954],
        [ 0.3030, -0.1494, -0.6886,  0.1698,  2.4682],
        [-0.8137,  0.4849, -1.2024, -0.4977,  0.1173]])
```

Output 3.13: A sliced tensor

Or if you use:

```
print(a[1:, 2:4])
```

Listing 3.18: Slicing a tensor in two dimensions

It will be:

```
tensor([[[ 0.3030, -0.1494, -0.6886,  0.1698,  2.4682],
         [-0.8137,  0.4849, -1.2024, -0.4977,  0.1173]],

        [[ 0.9666,  0.5375, -0.0313, -0.0679, -0.7232],
         [-0.5526,  0.9475, -0.5450,  0.0638,  1.2879]]])
```

Output 3.14: A sliced tensor

You can also make use of the same slicing syntax to *add* a new dimension. For example,

```
print(a[:, None, :, None].shape)
```

Listing 3.19: Adding a new dimension to a tensor with `None`

You will see:

```
torch.Size([3, 1, 4, 1, 5])
```

Output 3.15: A tensor with more dimensions added

Here you use `None` to insert a new dimension at a specific place. This is useful if, for example, you need to convert an image into a batch of only one image. If you're familiar with NumPy, you may recall there is a function `expand_dims()` for this purpose, but PyTorch doesn't provide it. A similar function is `unsqueeze()`, which is demonstrated below:

```
b = torch.unsqueeze(a, dim=2)
print(a.shape)
print(b.shape)
```

Listing 3.20: Adding a new dimension using `unsqueeze()`

This prints:

```
torch.Size([3, 4, 5])
torch.Size([3, 4, 1, 5])
```

Output 3.16: Tensor shape before and after adding a dimension

One powerful nature of NumPy slicing syntax is Boolean indexing. This is also supported with PyTorch tensors. For example:

```
import torch
a = torch.randn(3,4)
print(a)
print(a[:, (a > -1).all(axis=0)])
```

Listing 3.21: Boolean indexing on a tensor

You may see:

```
tensor([[ 1.2548,  0.4078,  0.5548, -0.7016],
        [-0.3720, -0.5682, -0.3430,  0.0886],
        [ 0.2151,  0.3626, -2.0275,  1.8121]])
tensor([[ 1.2548,  0.4078, -0.7016],
```

```
[-0.3720, -0.5682,  0.0886],
[ 0.2151,  0.3626,  1.8121]])
```

Output 3.17: Result of boolean indexing

The above selects the columns where all elements are greater than -1 . You can also manipulate the tensor by selecting specific columns:

```
print(a[:, [1,0,0,1]])
```

Listing 3.22: Creating a tensor by indexing on a dimension

This results in:

```
tensor([[ 0.4078,  1.2548,  1.2548,  0.4078],
        [-0.5682, -0.3720, -0.3720, -0.5682],
        [ 0.3626,  0.2151,  0.2151,  0.3626]])
```

Output 3.18: A tensor created by selecting columns from another tensor

To convert a 2D tensor into 1D, you can use:

```
import torch
a = torch.randn(3,4)
print(a)
print(a.ravel())
```

Listing 3.23: Flattening a 2D tensor to 1D

The result will be:

```
tensor([[ -0.2718, -0.8309,  0.6263, -0.2499],
        [-0.1780,  1.1735, -1.3530, -1.2374],
        [-0.6050, -1.5524, -0.1008, -1.2782]])
tensor([ -0.2718, -0.8309,  0.6263, -0.2499, -0.1780,  1.1735, -1.3530, -1.2374,
        -0.6050, -1.5524, -0.1008, -1.2782])
```

Output 3.19: The 2D tensor and its 1D equivalent

You may also use the `reshape()` function to achieve the same:

```
print(a.reshape(-1))
```

Listing 3.24: Reshape a tensor

The result should be the same as that of `ravel()`. But usually, the `reshape()` function is for more complicated target shapes:

```
print(a.reshape(3,2,2))
```

Listing 3.25: Reshape a tensor

This will print:

```
tensor([[[[-0.2718, -0.8309],
          [ 0.6263, -0.2499]],

        [[-0.1780,  1.1735],
          [-1.3530, -1.2374]],

        [[-0.6050, -1.5524],
          [-0.1008, -1.2782]]]])
```

Output 3.20: The tensor as reshaped

One common case of reshaping tensors is to do matrix transpose. For a 2D matrix, it is easily done in the same way as NumPy:

```
print(a.T)
```

Listing 3.26: Transpose a tensor

which prints:

```
tensor([[[-0.2718, -0.1780, -0.6050],
        [-0.8309,  1.1735, -1.5524],
        [ 0.6263, -1.3530, -0.1008],
        [-0.2499, -1.2374, -1.2782]])
```

Output 3.21: A tensor transposed

But the `transpose()` function in PyTorch, unlike NumPy, requires you to explicitly specify the two axes to swap:

```
print(a.transpose(0, 1))
```

Listing 3.27: Transpose a tensor as swapping axes

This result is same as above. If you have multiple tensors, you can combine them by stacking them (`vstack()` for vertically and `hstack()` for horizontally). For example:

```
import torch
a = torch.randn(3,3)
b = torch.randn(3,3)
print(a)
print(b)
print(torch.vstack([a,b]))
```

Listing 3.28: Stacking tensors

This may print:

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527]])
tensor([[ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527],
        [ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

Output 3.22: Stacked tensors

The concatenate function is similar:

```
c = torch.concatenate([a, b])
print(c)
```

Listing 3.29: Creating a new tensor by concatenating

You will get the same tensor:

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527],
        [ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

Output 3.23: Tensor created by `concatenate()` is same as `vstack()`

The reverse is to split, e.g.,

```
print(torch.vsplit(c, 2))
```

Listing 3.30: Splitting a tensor into two

It prints

```
(tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527]]), tensor([[ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]]))
```

Output 3.24: Two tensors as the result of splitting

This function tells how many tensors to split into, rather than what size each tensor is. The latter is indeed more useful in deep learning (e.g., to split a tensor of a large dataset into many tensors of small batches). The equivalent function would be:

```
print(torch.split(c, 3, dim=0))
```

Listing 3.31: Splitting a tensor on a particular dimension

This should give you the same result as before. So `split(c, 3, dim=0)` means to split on dimension 0 such that each resulting tensor will be of size 3.

3.4 Tensor Functions

PyTorch tensors can be treated as arrays. So you can often use it in a similar way as NumPy arrays. For example, you have the functions of common mathematical functions:

```
import torch
a = torch.randn(2,3)
print(a)
print("exp =", torch.exp(a))
print("log =", torch.log(a))
print("sin =", torch.sin(a))
print("arctan =", torch.arctan(a))
print("abs =", torch.abs(a))
print("square =", torch.square(a))
print("sqrt =", torch.sqrt(a))
print("ceil =", torch.ceil(a))
print("round =", torch.round(a))
print("clip =", torch.clip(a, 0.1, 0.9))
```

Listing 3.32: Some tensor functions in PyTorch

This prints:

```
tensor([[ 1.2001,  1.0952,  1.1952],
        [ 0.1955, -2.8078,  0.3427]])
exp = tensor([[3.3206, 2.9897, 3.3042],
              [1.2159, 0.0603, 1.4087]])
log = tensor([[ 0.1824,  0.0909,  0.1783],
              [-1.6323,   nan, -1.0709]])
sin = tensor([[ 0.9321,  0.8890,  0.9303],
              [ 0.1942, -0.3277,  0.3360]])
arctan = tensor([[ 0.8761,  0.8308,  0.8741],
                 [ 0.1930, -1.2286,  0.3302]])
abs = tensor([[1.2001, 1.0952, 1.1952],
              [0.1955, 2.8078, 0.3427]])
square = tensor([[1.4403, 1.1994, 1.4285],
                 [0.0382, 7.8835, 0.1174]])
sqrt = tensor([[1.0955, 1.0465, 1.0932],
               [0.4421,   nan, 0.5854]])
ceil = tensor([[ 2.,  2.,  2.],
               [ 1., -2.,  1.]])
round = tensor([[ 1.,  1.,  1.],
                [ 0., -3.,  0.]])
clip = tensor([[0.9000, 0.9000, 0.9000],
               [0.1955, 0.1000, 0.3427]])
```

Output 3.25: Result of the tensor functions

Note that if a function is undefined (e.g., square root of negative numbers), `nan` will be the result, but no exception will be raised. In PyTorch, you have a function to check if the values of a tensor are `nan`:

```
import torch
b = torch.sqrt(a)
print(b)
print(torch.isnan(b))
```

Listing 3.33: Finding which value of a tensor is nan

You will get:

```
tensor([[1.0280,  nan,  nan],
        [ nan, 1.1473,  nan]])
tensor([[False,  True,  True],
        [ True, False,  True]])
```

Output 3.26: A Boolean tensor indicating if a value in a tensor is nan

Indeed, besides these defined functions, the Python operators can be applied to the tensors too:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(2, 3)
print(a)
print(b)
print(a+b)
print(a/b)
print(a ** 2)
```

Listing 3.34: Tensor operations

You get:

```
tensor([[ 0.7378, -0.3469,  1.3089],
        [-1.9152,  0.3745, -0.7248]])
tensor([[ -0.3650, -0.4768,  0.9331],
        [ 0.5095,  1.7169, -0.5463]])
tensor([[ 0.3729, -0.8237,  2.2421],
        [-1.4058,  2.0914, -1.2711]])
tensor([[ -2.0216,  0.7275,  1.4027],
        [-3.7594,  0.2181,  1.3269]])
tensor([[0.5444, 0.1203, 1.7133],
        [3.6682, 0.1403, 0.5254]])
```

Output 3.27: Result of tensor operations

But among the operators, matrix multiplications are very important in deep learning. You can do this with:

```
print(torch.matmul(a, b.T))
print(a @ b.T)
```

Listing 3.35: Matrix multiplications

This prints

```
tensor([[ 1.1176, -0.9347],
        [-0.1560,  0.0632]])
tensor([[ 1.1176, -0.9347],
        [-0.1560,  0.0632]])
```

Output 3.28: Result of matrix multiplication

These two are the same. Indeed, the @ operator from Python can also be used for vector dot-product, e.g.:

```
import torch
a = torch.randn(3)
b = torch.randn(3)
print(a)
print(b)
print(torch.dot(a, b))
print(a @ b)
```

Listing 3.36: Computing dot-product in PyTorch

It prints:

```
tensor([-0.8986, -0.6994,  1.1443])
tensor([-1.0666,  0.1455,  0.1322])
tensor(1.0081)
tensor(1.0081)
```

Output 3.29: Result of dot-product

If you treat the values in a tensor as samples, you may also want to find some statistics about it. Some are provided in PyTorch too:

```
import torch
a = torch.randn(3,4)
print(a)
print(torch.mean(a, dim=0))
print(torch.std(a, dim=0))
print(torch.cumsum(a, dim=0))
print(torch.cumprod(a, dim=0))
```

Listing 3.37: Basic statistics on a tensor

It prints:

```
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
        [-0.5712,  0.8430, -1.6147, -1.1664],
        [ 1.7298, -1.7665, -0.5918,  0.3024]])
tensor([ 0.4972, -0.3142, -0.5750, -0.6708])
tensor([1.1593, 1.3295, 1.0482, 0.8429])
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
        [-0.2381,  0.8240, -1.1333, -2.3148],
        [ 1.4917, -0.9425, -1.7251, -2.0124]])
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
```

```
[-0.1903, -0.0160, -0.7774,  1.3395],
[-0.3291,  0.0283,  0.4601,  0.4051]])
```

Output 3.30: Result of some basic statistics on a tensor

But for linear algebra functions, you should find it in PyTorch's `linalg` submodule. For example:

```
print(torch.linalg.svd(a))
```

Listing 3.38: Using the `linalg` submodule

You will see:

```
torch.return_types.linalg_svd(
U=tensor([[ -0.0353,  0.1313,  0.9907],
          [-0.5576,  0.8201, -0.1286],
          [ 0.8294,  0.5569, -0.0443]]),
S=tensor([2.7956, 1.9465, 1.2715]),
Vh=tensor([[ 0.6229, -0.6919,  0.1404,  0.3369],
           [ 0.2767, -0.1515, -0.8172, -0.4824],
           [ 0.2570, -0.0385,  0.5590, -0.7874],
           [ 0.6851,  0.7048, -0.0073,  0.1840]]))
```

Output 3.31: Result of singular value decomposition on a tensor

And specifically for convolution neural networks, padding a tensor is done with the following:

```
b = torch.nn.functional.pad(a, (1,1,0,2), value=0)
print(b)
```

Listing 3.39: Padding a tensor with a value

This prints:

```
tensor([[ 0.0000,  0.3331, -0.0190,  0.4814, -1.1484,  0.0000],
        [ 0.0000, -0.5712,  0.8430, -1.6147, -1.1664,  0.0000],
        [ 0.0000,  1.7298, -1.7665, -0.5918,  0.3024,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]])
```

Output 3.32: A padded tensor

This example of the `pad()` function is to create (1,1) padding on dimension 0 and (0,2) on dimension 1. In other words, for each dimension 0 (rows), we add one dummy value (0) at the beginning and the end. For each dimension 1 (columns), we add zero dummy values at the beginning but two dummy values at the end.

Finally, since PyTorch tensors can be considered arrays, you can use them directly with other tools such as `matplotlib`. Below is an example of plotting a surface using PyTorch tensors:

```

import matplotlib.pyplot as plt
import torch

# create tensors
x = torch.linspace(-1, 1, 100)
y = torch.linspace(-2, 2, 100)
# create the surface
xx, yy = torch.meshgrid(x, y, indexing="xy") # xy-indexing is matching numpy
z = torch.sqrt(1 - xx**2 - (yy/2)**2)
print(xx)

fig = plt.figure(figsize=(8,8))
ax = plt.axes(projection="3d")
ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])
ax.set_zlim([0, 2])
ax.plot_surface(xx, yy, z, cmap="cividis")
ax.view_init(45, 35)
plt.show()

```

Listing 3.40: Plotting a surface using matplotlib from a PyTorch tensor

The mesh grid produced the tensor `xx` as:

```

tensor([[ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        ...,
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000]])

```

Output 3.33: Result of `meshgrid()` function

Which values are identical along the second dimension. Similarly for the tensor `yy`, except the repetition is on the other dimension. The plot created is:

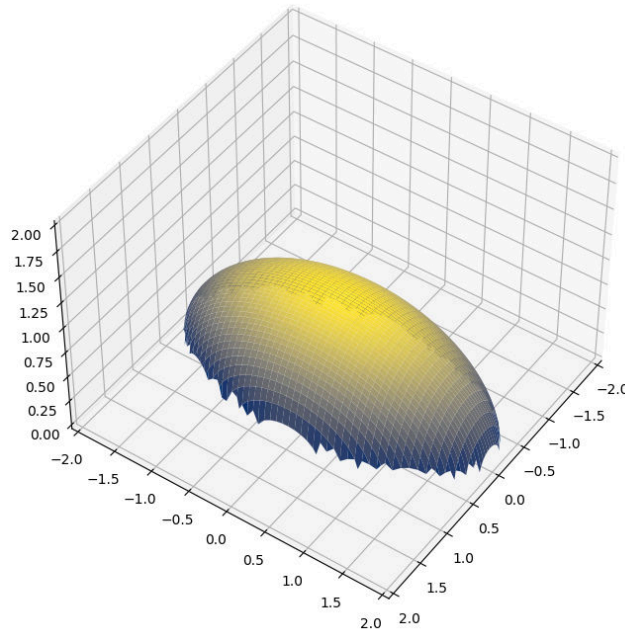


Figure 3.1: Surface plot from matplotlib

3.5 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

Tensors. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

Introduction to PyTorch Tensors. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/introyt/tensors_deeper_tutorial.html

3.6 Summary

In this chapter, you discovered how to manipulate PyTorch tensors. Specifically, you learned:

- ▷ What is a tensor
- ▷ How to create various kinds of tensors in PyTorch
- ▷ How to reshape, slice, and manipulate tensors in PyTorch
- ▷ The common functions that can be applied to PyTorch tensors

In the next chapter, you will learn how to use the automatic differentiation feature in PyTorch to solve some simple problems.

Using Autograd in PyTorch to Solve a Regression Problem

We usually use PyTorch to build a neural network. However, PyTorch can do more than this. Because PyTorch is also a tensor library with automatic differentiation capability, you can easily use it to solve a numerical optimization problem with gradient descent. In this chapter, you will learn how PyTorch's automatic differentiation engine, autograd, works.

After finishing this chapter, you will learn:

- ▷ What is autograd in PyTorch
- ▷ How to make use of autograd and an optimizer to solve an optimization problem

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Autograd in PyTorch
- ▷ Using Autograd for Polynomial Regression
- ▷ Using Autograd to Solve a Math Puzzle

4.1 Autograd in PyTorch

In PyTorch, you can create tensors as variables or constants and build an expression with them. The expression is essentially a function of the variable tensors. Therefore, you may derive its derivative function, i.e., the differentiation or the gradient. This is the foundation of the training loop in a deep learning model. PyTorch comes with this feature at its core.

It is easier to explain autograd with an example. In PyTorch, you can create a constant matrix as follows:

```
import torch

x = torch.tensor([1, 2, 3])
print(x)
```

```
print(x.shape)
print(x.dtype)
```

Listing 4.1: Setting up a constant matrix in PyTorch

The above prints:

```
tensor([1, 2, 3])
torch.Size([3])
torch.int64
```

Output 4.1: The constant tensor created

This creates an integer vector (in the form of a PyTorch tensor). This vector can work like a NumPy vector in most cases. For example, you can do `x+x` or `2*x`, and the result is just what you would expect. PyTorch comes with many functions for array manipulation that match NumPy, such as `torch.transpose` or `torch.concatenate`.

But this tensor is not assumed to be a variable for a function in the sense that differentiation with it is not supported. You can create tensors that work like a variable with an extra option:

```
import torch

x = torch.tensor([1., 2., 3.], requires_grad=True)
print(x)
print(x.shape)
print(x.dtype)
```

Listing 4.2: Setting up a tensor to support differentiation

This will print:

```
tensor([1., 2., 3.], requires_grad=True)
torch.Size([3])
torch.float32
```

Output 4.2: A tensor to support differentiation

Note that, in the above, a tensor of floating point values was created. It is required because differentiation requires floating points, not integers.

The operations (such as `x+x` and `2*x`) can still be applied, but in this case, the tensor will remember how it got its values. You can demonstrate this feature in the following:

```
import torch

x = torch.tensor(3.6, requires_grad=True)
y = x * x
y.backward()
print("x =", x)
print("y =", y)
print("x.grad =", x.grad)
```

Listing 4.3: Tensor operations

This prints:

```
x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)
```

Output 4.3: Tensor operation

What it does is the following: This defined a variable x (with value 3.6) and then computed $y=x*x$ or $y = x^2$. Then you ask for the differentiation of y . Since y obtained its value from x , you can find the derivative $\frac{dy}{dx}$ at $x.grad$, in the form of a tensor, immediately after you run `y.backward()`. You know $y = x^2$ means $y' = 2x$. Hence the output would give you a value of $3.6 \times 2 = 7.2$.

4.2 Using Autograd for Polynomial Regression

How is this feature in PyTorch helpful? Let's consider a case where you have a polynomial in the form of $y = f(x)$, and you are given several (x, y) samples. How can you recover the polynomial $f(x)$? One way is to assume a random coefficient for the polynomial and feed in the samples (x, y) . If the polynomial is found, you should see the value of y matches $f(x)$. The closer they are, the closer your estimate is to the correct polynomial.

This is indeed a numerical optimization problem where you want to minimize the difference between y and $f(x)$. You can use gradient descent to solve it.

Let's consider an example. You can build a polynomial $f(x) = x^2 + 2x + 3$ in NumPy as follows:

```
import numpy as np

polynomial = np.poly1d([1, 2, 3])
print(polynomial)
```

Listing 4.4: Creating a polynomial in NumPy

This prints:

```
2
1 x + 2 x + 3
```

Output 4.4: A polynomial

You may use the polynomial as a function, such as:

```
print(polynomial(1.5))
```

Listing 4.5: Using a NumPy polynomial as a function

And this prints 8.25, for $(1.5)^2 + 2 \times (1.5) + 3 = 8.25$.

Now you can generate a number of samples from this function using NumPy:

```

N = 20    # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)

```

Listing 4.6: Generating samples from a polynomial

In the above, both X and Y are NumPy arrays of the shape $(20,1)$, and they are related as $y = f(x)$ for the polynomial $f(x)$.

Now, assume you do not know what the polynomial is except it is quadratic. And you want to recover the coefficients. Since a quadratic polynomial is in the form of $Ax^2 + Bx + C$, you have three unknowns to find. You can find them using the gradient descent algorithm you implement or an existing gradient descent optimizer. The following demonstrates how it works:

```

import torch

# Assume samples X and Y are prepared elsewhere

XX = np.hstack([X*X, X, np.ones_like(X)])

w = torch.randn(3, 1, requires_grad=True) # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32) # input sample
y = torch.tensor(Y, dtype=torch.float32)  # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

for _ in range(1000):
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    optimizer.zero_grad()
    mse.backward()
    optimizer.step()

print(w)

```

Listing 4.7: Gradient descent in PyTorch

The `print` statement before the `for` loop gives three random numbers, such as:

```

tensor([[1.3827],
        [0.8629],
        [0.2357]], requires_grad=True)

```

Output 4.5: The three random numbers from the initialized tensor

But the one after the `for` loop gives you the coefficients very close to that in the polynomial:

```

tensor([[1.0004],
        [1.9924],
        [2.9159]], requires_grad=True)

```

Output 4.6: The solution produced by gradient descent

What the above code does is the following: First, it creates a variable vector w of 3 values, namely the coefficients A, B, C . Then you create an array of shape $(N, 3)$, in which N is the number of samples in the array X . This array has 3 columns: the values of x^2 , x , and 1, respectively. Such an array is built from the vector X using the `np.hstack()` function. Similarly, you build the tensor y from the NumPy array Y .

Afterward, you use a for loop to run the gradient descent in 1,000 iterations. In each iteration, you compute $x \times w$ in matrix form to find $Ax^2 + Bx + C$ and assign it to the variable `y_pred`. Then, compare y and `y_pred` and find the mean square error. Next, derive the gradient, i.e., the rate of change of the mean square error with respect to the coefficients w using the `backward()` function. And based on this gradient, you use gradient descent to update w via the optimizer.

In essence, the above code will find the coefficients w that minimizes the mean square error.

Putting everything together, the following is the complete code:

```
import numpy as np
import torch

polynomial = np.poly1d([1, 2, 3])
N = 20 # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)

# Prepare input as an array of shape (N,3)
XX = np.hstack([X*X, X, np.ones_like(X)])

# Prepare tensors
w = torch.randn(3, 1, requires_grad=True) # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32) # input sample
y = torch.tensor(Y, dtype=torch.float32) # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

# Run optimizer
for _ in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()

print(w)
```

Listing 4.8: Using gradient descent and automatic differentiation to fit a polynomial

4.3 Using Autograd to Solve a Math Puzzle

In the above, 20 samples were used, which is more than enough to fit a quadratic equation. You may use gradient descent to solve some math puzzles as well. For example, the following problem:

$$\begin{array}{rclcl} A & + & B & = & 9 \\ + & & - & & \\ C & - & D & = & 1 \\ = & & = & & \\ 8 & & 2 & & \end{array}$$

In other words, to find the values of A, B, C, D such that:

$$A + B = 9$$

$$C - D = 1$$

$$A + C = 8$$

$$B - D = 2$$

This can also be solved using autograd, as follows:

```
import random
import torch

A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)

# Gradient descent loop
EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)
for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2
    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print(A)
print(B)
print(C)
print(D)
```

Listing 4.9: Solving a math puzzle with autograd in PyTorch

There can be multiple solutions to this problem. One solution is the following:

```
tensor(4.7191, requires_grad=True)
tensor(4.2808, requires_grad=True)
tensor(3.2808, requires_grad=True)
tensor(2.2808, requires_grad=True)
```

Output 4.7: Solution to the math puzzle

Which means $A = 4.72$, $B = 4.28$, $C = 3.28$, and $D = 2.28$. You can verify this solution fits the problem.

The above code defines the four unknowns as variables with a random initial value. Then you compute the result of the four equations and compare it to the expected answer. You then sum up the squared error and ask PyTorch's optimizer to minimize it. The minimum possible square error is zero, attained when our solution exactly fits the problem.

Note the way PyTorch produces the gradient: You ask for the gradient of `sqerr`, which it noticed that, among other things, only `A`, `B`, `C`, and `D` are its dependencies that `requires_grad=True`. Hence four gradients are found. You then apply each gradient to the respective variables in each iteration via the optimizer.

4.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

Autograd mechanics. PyTorch documentation.

<https://pytorch.org/docs/stable/notes/autograd.html>

Automatic differentiation package — torch.autograd. PyTorch API.

<https://pytorch.org/docs/stable/autograd.html>

4.5 Summary

In this chapter, we demonstrated how PyTorch's automatic differentiation works. This is the building block for carrying out deep learning training. Specifically, you learned:

- ▷ What is automatic differentiation in PyTorch
- ▷ How you can use gradient tape to carry out automatic differentiation
- ▷ How you can use automatic differentiation to solve an optimization problem

Starting the next chapter, you will learn about the major application of PyTorch — the deep learning models.